Performance models of parallel programs

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

Credits

- prof. David Padua
  - https://courses.engr.illinois.edu/cs420/fa2012/lectures.html
- Samuel Williams, LBL
  - https://amcr.lbl.gov/wp-content/uploads/2025/11/roofline-intro.pdf

Scalability

- How much faster can a given problem be solved with *p* workers instead of one?
- How much more work can be done with *p* workers instead of one?
- What impact for the communication requirements of the parallel application have on performance?
- What fraction of the resources is actually used productively for solving the problem?

The Work/Span model

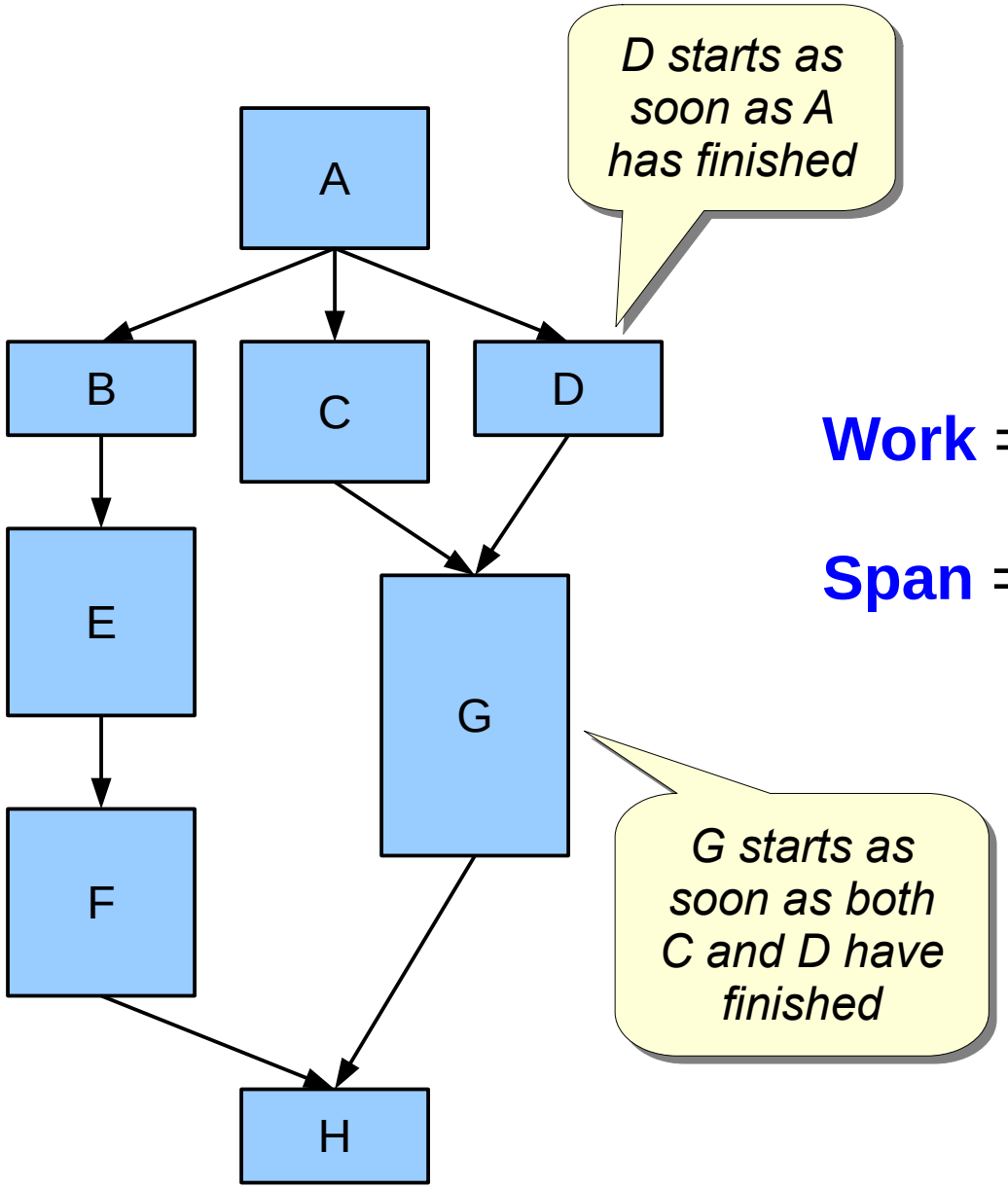
Work/Span model

- Assume
  - n input size.
  - p number of execution units.
  - $T_p(n)$ wall-clock time of the program with p processors and input size n.
  - Shared memory model (can be extended to distributed memory with message passing).
  - Execution units are fully independent and asynchronous.

Work/Span model

- **Work** $T_1(n)$
  - Total number of operations performed by all execution units.
  - i.e., the computational cost of the sequential algorithm.
- **Span** $T_p(n)$
  - Number of operations executed along the critical path (longest chain of sequential steps).
- **Cost** $p\ T_p(n)$
  - Total time spent by all processors during the computation either working or waiting.
- **Overhead** $p\ T_p(n) - T_1(n)$
  - Cost – Work.

Example

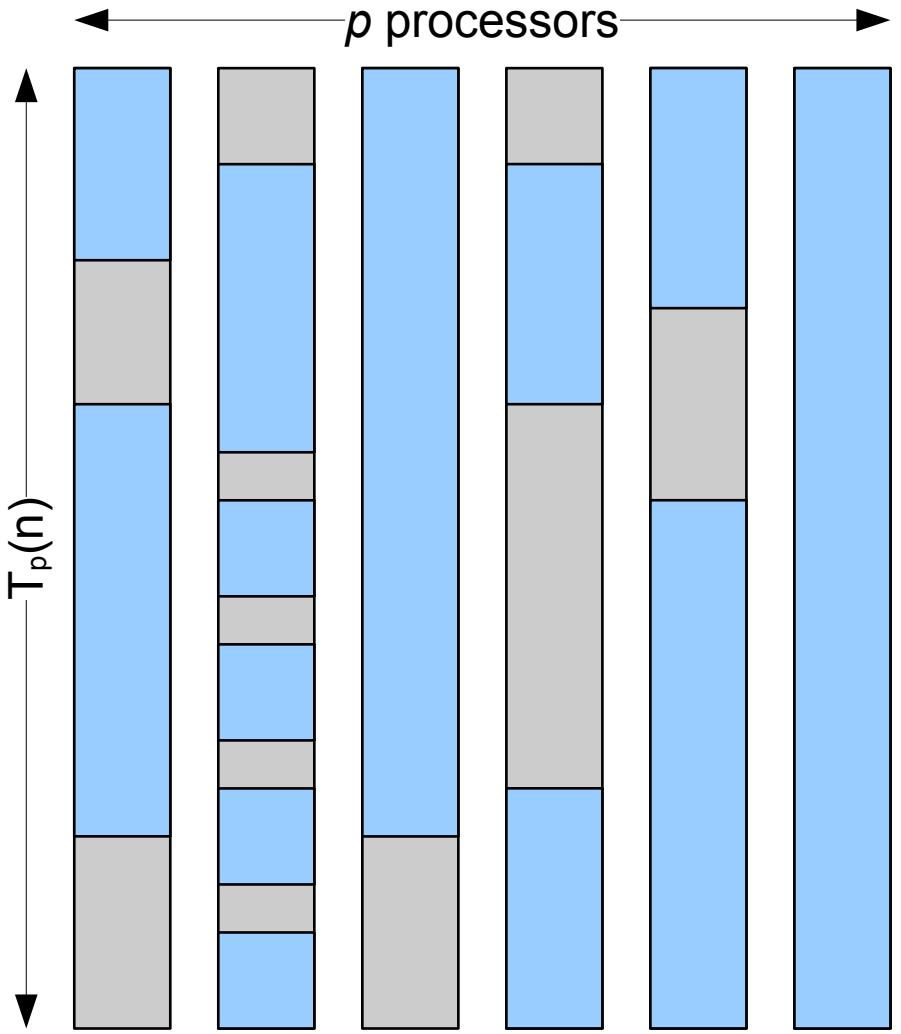


Work = $A + B + C + D + E + F + G + H$

Span = max{ $A + B + E + F + H$,
$A + C + G + H$,
$A + D + G + H$ }

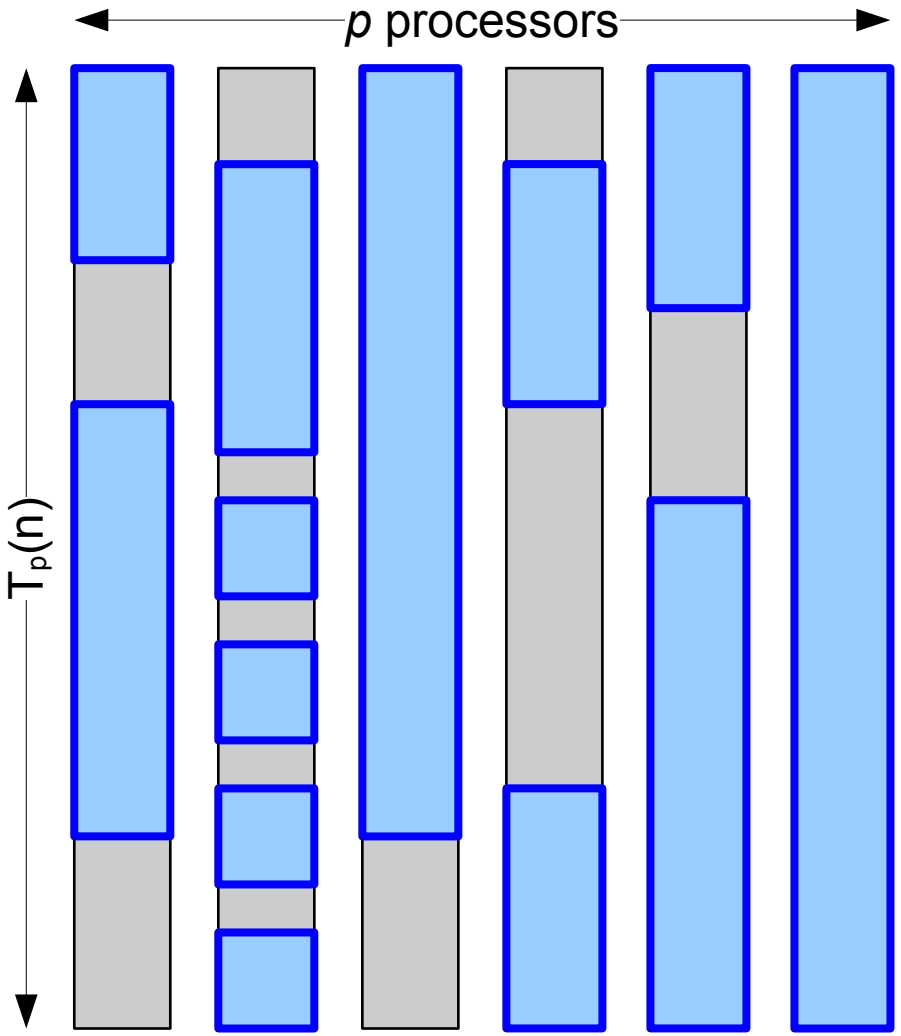
Work/Span cost model

- Visualize a parallel computation as a set of alternating busy[blue] / idle[grey] periods.
- The length of blue bars is proportional to the number of operations performed during the corresponding period.



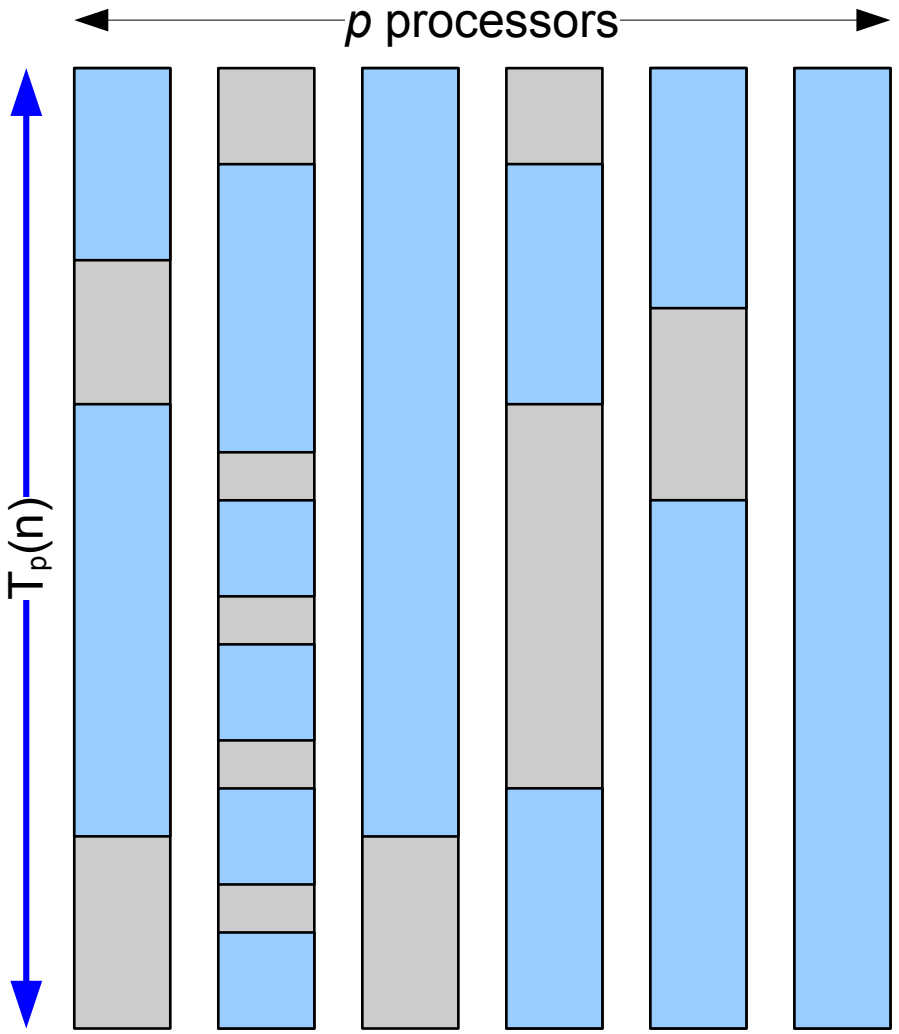
Work/Span cost model

- Work = $T_1(n)$
  - Total number of operations in all blue boxes.



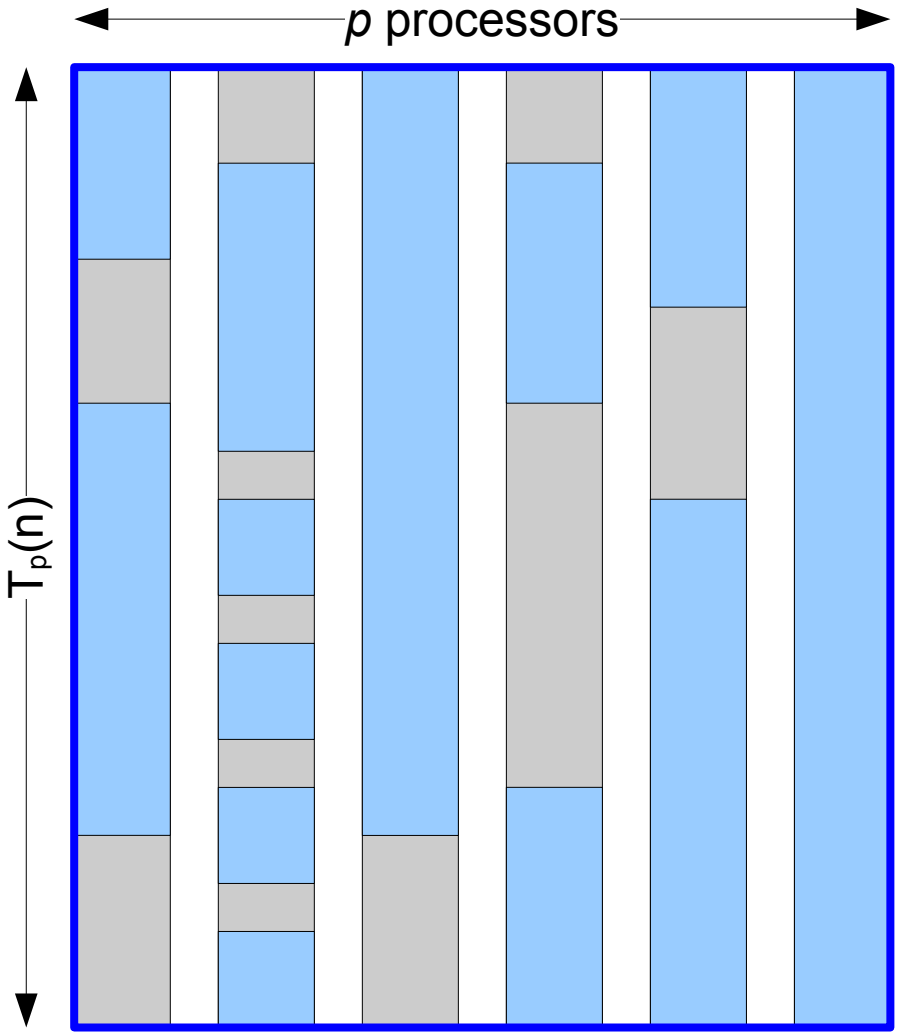
Work/Span cost model

- Span = $T_p(n)$
  - Duration (Wall-Clock Time) of the parallel computation.
  - The time elapsed from the beginning of the earliest "busy" period to the end of the last one.



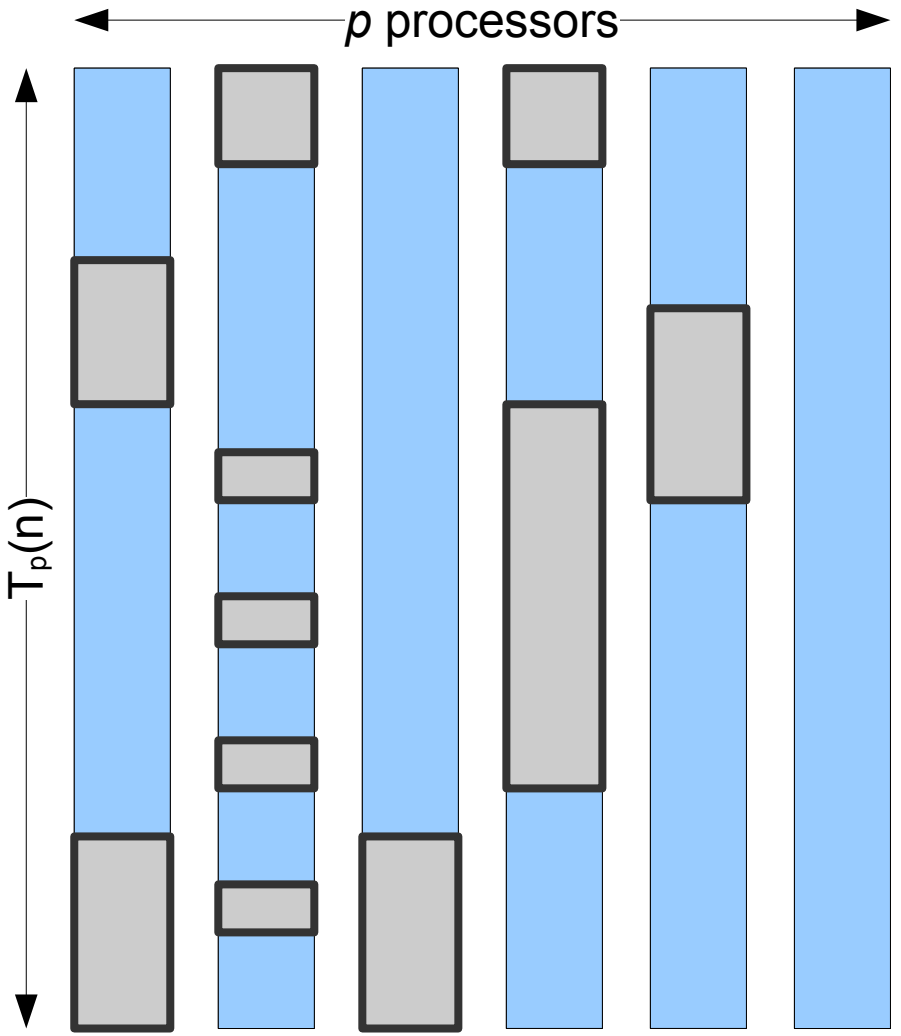
Work/Span cost model

- Cost = $p\ T_p(n)$
  - Sum of lengths of all lines from beginning to end, including idle periods.



Work/Span cost model

- Overhead = $p\ T_p(n) - T_1(n)$
  - Sum of lengths of all grey boxes.



Work and Span laws

- **Work law**: $p\,T_p(n) \geq T_1(n)$
  - Proof: p processors can execute at most p operations at each time step, in parallel.
- **Span law**: $T_p(n) \geq T_\infty(n)$
  - A finite number of processors cannot be faster than an infinite number of them.

Derived metrics: Speedup

- **Speedup**

$$S(p,n) = \frac{T_1(n)}{T_p(n)}$$

- From the work law $p\,T_p(n) \geq T_1(n)$ we have:

$$S(p,n) = \frac{T_1(n)}{T_p(n)} \leq \frac{pT_p(n)}{T_p(n)} = p$$

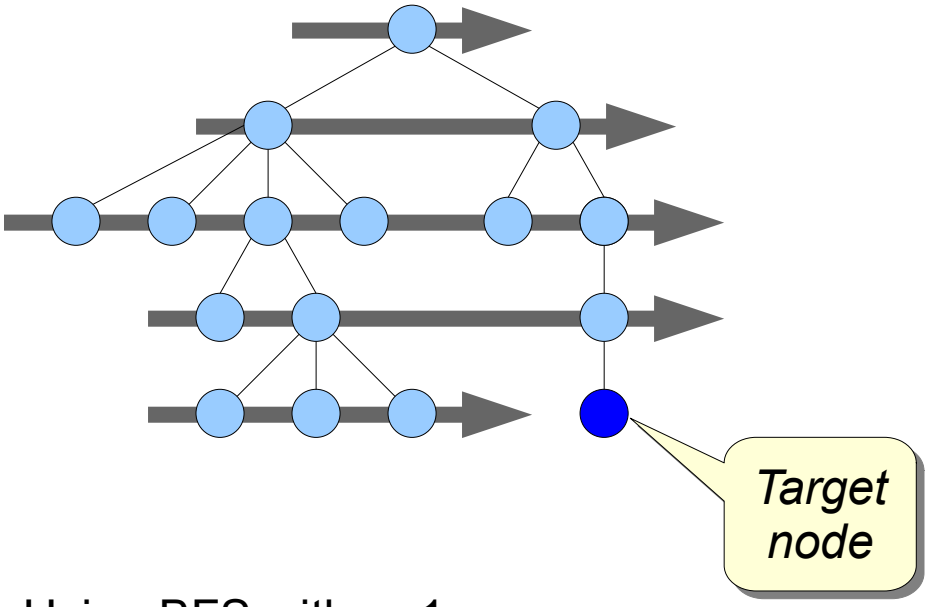
therefore $S(p, n) \leq p$

Derived metrics: Speedup

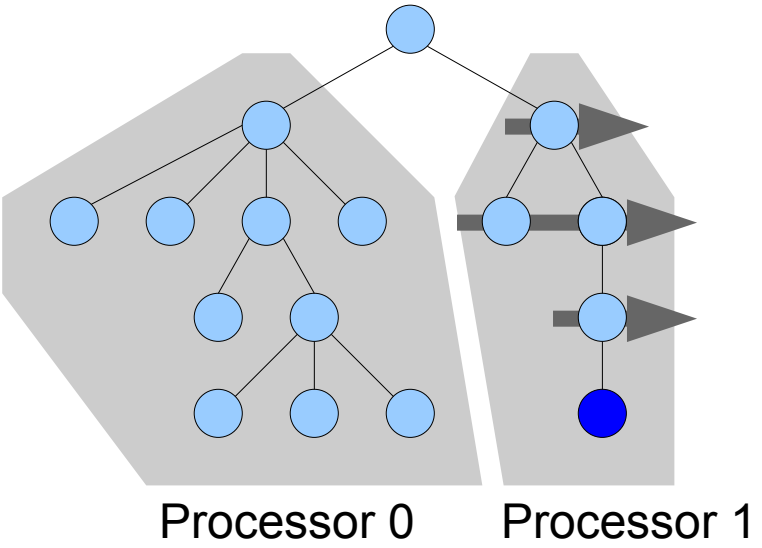
- Usually the speedup is computed by keeping n fixed and increasing the number of processors p.
- Ideally, we want a speedup as high as possible
- Most of the time, $S(p, n) < p$.
  - Due to synchronization overhead, scheduling overhead, algorithmic inefficiencies, etc.
- Rarely we might observe $S(p, n) > p$ (*superlinear speedup*).
  - Increasing the number of processes might decrease the working set on each process, improving cache utilization.
  - Some algorithms benefit from *exploratory decomposition* (see next slide)

Superlinear speedup by exploratory decomposition

- Suppose we want to search for a particular value on an unordered tree.



Using BFS with *p*=1 processor, we need to explore 15 nodes before reaching the target.



Using BFS with *p*=2 processors, splitting the tree at the root, processor 1 needs to explore only 4 nodes before reaching the target.

Derived metrics: Strong Scaling Efficiency

- **Strong Scaling Efficiency**

$$E(p,n) = \frac{S(p,n)}{p} = \frac{T_1(n)}{pT_p(n)}$$

- From $S(p) \leq p$, we get $E(p, n) \leq 1$

- High efficiency means that we are making good use of processing resources.

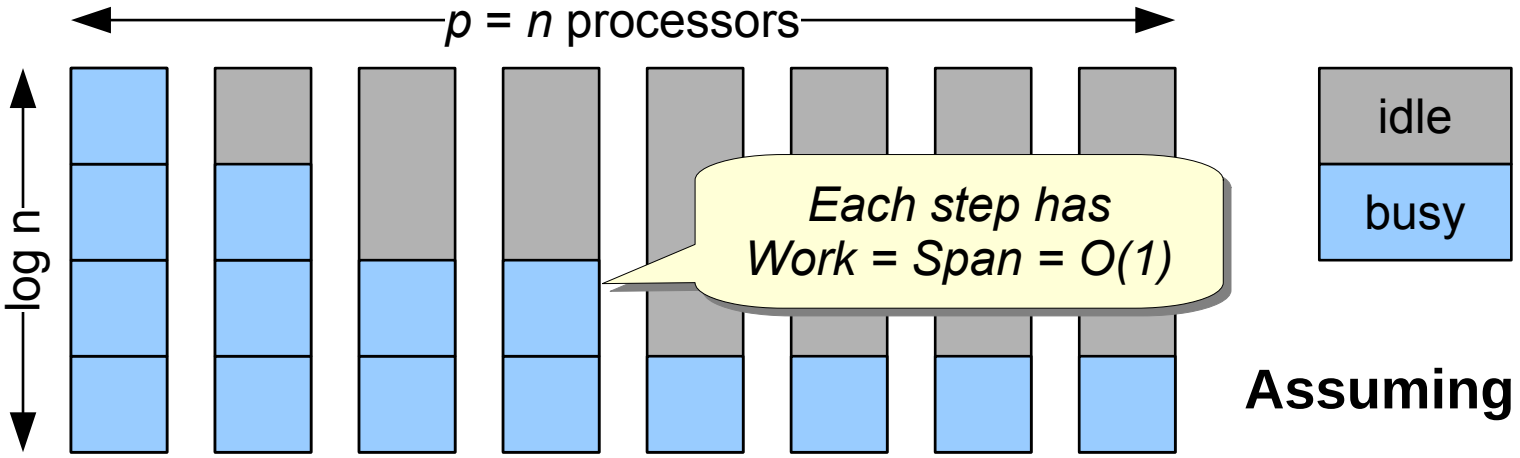
Examples

Example: Tree Sum-Reduction

```
float SumReduce( const float v[], int i, int j )
{
    if (i > j)
        return 0;
    else if (i == j)
        return v[i];
    else {
        const int m = (i + j) / 2;
        int s1, s2;
#pragma omp task shared(s1)
        s1 = SumReduce(v, i, m);
#pragma omp task shared(s2)
        s2 = SumReduce(v, m+1, j);
#pragma omp taskwait
        return s1 + s2;
    }
}
```

```
/* Invocation */
float s;
float *v = ...
#pragma omp parallel
#pragma omp single
s = SumReduce(v, 0, n-1);
```

Analysis (intuition)




```
float SumReduce( const float v[], int i, int
j )
{
     if (i > j)
          return 0;
     else if (i == j)
          return v[i];
     else {
          const int m = (i + j) / 2;
          int s1, s2;
#pragma omp task shared(s1)
          s1 = SumReduce(v, i, m);
#pragma omp task shared(s2)
          s2 = SumReduce(v, m+1, j);
#pragma omp taskwait
          return s1 + s2;
     }
}
```

Assuming *p* = *n* processors

Work = $T_1(n)$ =
1 + 2 + 4 + … + n = O(n)

Span = $T_n(n)$ =
height of tree = O(log(n))

Speedup = $T_1(n)$ / $T_p(n)$ =
O(n / log(n))

Efficiency = Speedup / p =
O(1 / log(n))

Analysis (better)

```
float SumReduce( const float v[], int i, int j )
{
    if (i > j)
        return 0;
    else if (i == j)
        return v[i];
    else {
        const int m = (i + j) / 2;
        int s1, s2;
#pragma omp task shared(s1)
        s1 = SumReduce(v, i, m);
#pragma omp task shared(s2)
        s2 = SumReduce(v, m+1, j);
#pragma omp taskwait
        return s1 + s2;
    }
}
```

Assuming p = ∞ processors

Work: $W(n) = 2\,W(n/2) + O(1) = n$

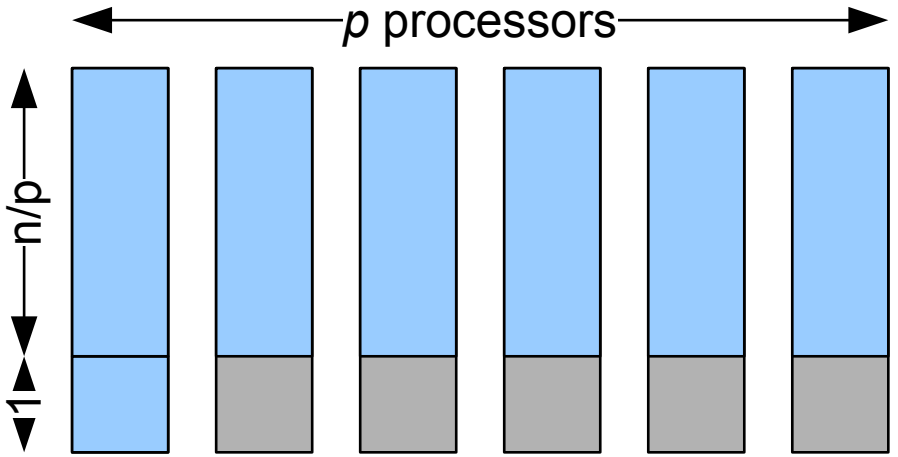
Span: $S(n) = S(n/2) + O(1) = \log n$

Sum-reduction with *p* processors

```
float SumReduce( const float v[], int n )
{
    float result = 0.0;
#pragma omp parallel for reduction(+:result)
    for (int i=0; i<n; i++) {
        result += v[i];
    }
    return result;
}
```

- Assumptions:
  - *p* processors, *p* << *n.*
  - Static scheduling of for loop.
  - Reduction requires O(1) time.

Sum-reduction with *p* processors and O(1) reduction



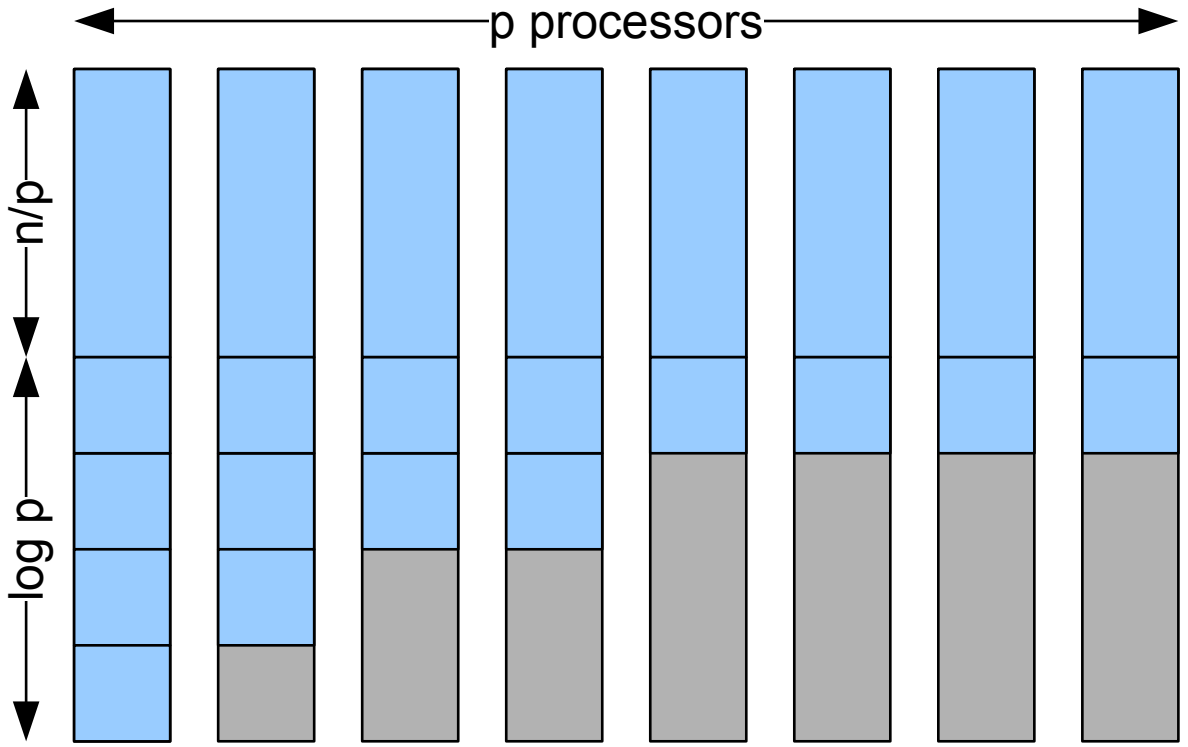
- **Work** = $T_1(n)$ = $p\ (n/p) + O(1) = n$
- **Span** = $T_p(n)$ = $n/p + O(1) = n/p$
- **Speedup** = $T_1(n) / T_p(n)$ = p
- **Efficiency** = $S_p(n) / p$ = 1

Sum-reduction with *p* processors and O(log *p*) reduction

```
float SumReduce( const float v[], int n )
{
    float result = 0.0;
#pragma omp parallel for reduction(+:result)
    for (int i=0; i<n; i++) {
        result += v[i];
    }
    return result;
}
```

- Assumptions:
  - *p* << *n* processors.
  - Static scheduling of for loop.
  - A reduction requires work *p* and span log *p* at the end of the "for" loop.

Parallel version with *p* processors



- **Work** = $T_1(n) = n + p$ = n if $p = O(n)$
- **Span** = $T_p(n) = n/p + \log p$ = n/p if $(p \log p) = O(n)$
- **Speedup** = $T_1(n) / T_p(n)$ = p if $(p \log p) = O(n)$
- **Efficiency** = Speedup / p = 1 if $(p \log p) = O(n)$

Measuring scalability, empirically

Estimating $T_p(n)$

- We set the input input size n to some fixed value
  - The input size should be large enough such that the execution time of our algorithm is not too small.
- We define $T_p(n)$ as the measured Wall-Clock Time (WCT) of the parallel program executed with $p = 1, \ldots P$ processors.
- From $T_p(n)$ we compute the speedup and strong scaling efficiency as a function of p using the respective definitions.

Taking times

- To compute speedup and efficiencies one needs to take the program wall clock time, excluding the input/output time.

***Must be** at the **very beginning** before any #include statement (can be omitted for CUDA)*

The actual computation is done here

```
#if _XOPEN_SOURCE < 600
#define _XOPEN_SOURCE 600
#endif
#include "hpc.h"
other #includes here...
...
double start, finish;
/* read input; this should not be measured */

start = hpc_gettime();
/* actual code that we want to time */
finish = hpc_gettime();

printf("Elapsed time = %e seconds\n", finish - start);

/* write output; this should not be measured */
```

How to measure the wall-clock time

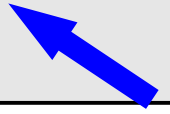
- Using OpenMP: **`omp_get_wtime()`**
- Using MPI: **`MPI_Wtime()`**
- Generic solution: **`clock_gettime()`**
- **Wrong solution: `clock()`**

```
NAME
       clock - Determine processor time

SYNOPSIS
       #include <time.h>

       clock_t clock(void);

DESCRIPTION
       The clock() function returns an
approximation of processor time used by the
program.
```



Types of scaling efficiencies

- *Strong Scaling*: increase the number of processors *p* keeping the *total* problem size fixed.
  - The *total* amount of work remains constant.
  - The amount of work for a single processor decreases as *p* increases.
  - Goal: reduce the total execution time by adding more processors.
- *Weak Scaling*: increase the number of processors *p* keeping the *per-processor* work fixed.
  - The total amount of work grows as *p* increases.
  - Goal: solve larger problems within the same amount of time.

Strong and Weak Scaling Efficiency

- *E(p)* = Strong Scaling Efficiency.
  - Fix the input size *n*, and compute for each *p* = 1, ... *P*

$$E(p) = \frac{S(p)}{p} = \frac{T_1(n)}{p \times T_p(n)}$$

- *W(p)* = Weak Scaling Efficiency.
  - Fore each *p* = 1, ... *P* compute the input size n_p (see next slide), then

$$W(p) = \frac{T_1(n_1)}{T_p(n_p)}$$

Computing n_p

- To compute $W(p)$ we need to increase the input size n_p in such a way that the amount of work done by each processor remains constant.
- Let $f(n_p, p)$ be the amount of work done by each processor, given input size n_p and p processors
- The input size(s) n_p must satisfy:

$$f(n_p, p) = \text{constant}$$

Example 1: Array sum

- f(n_p, p) = amount of work performed by each processor.
- Here we have:
  - f(n_p, p) = n_p / p
- Hence
  - f(n_p, p) = *const*
    n_p / p = *const*
    n_p = $p \times$ *const*

```
#pragma omp parallel for
for (int i=0; i<n; i++)
    c[i] = a[i] + b[i];
```

Example 2: Matrix addition

- f(n_p, p) = amount of work performed by each processor.
- Here we have:
  - f(n_p, p) = n_p^2 / p
- Hence:
  - f(n_p, p) = *const*
    n_p^2 / p = *const*
    n_p = sqrt(p × *const*)
    n_p = sqrt(p) × *const'*

```
#pragma omp parallel for
for (int i=0; i<n; i++) {
  for (int j=0; j<n; j++) {
    C[i][j] = A[i][j] + B[i][j];
```

Example 3: Matmul

- f(n_p, p) = amount of work performed by each processor.
- Here we have:
  - f(n_p, p) = n_p^3 / p
- Hence:
  - f(n_p, p) = *const*
    $n_p^3 / p = const$
    $n_p = (p \times const)^{1/3}$
    $n_p = p^{1/3} \times const'$

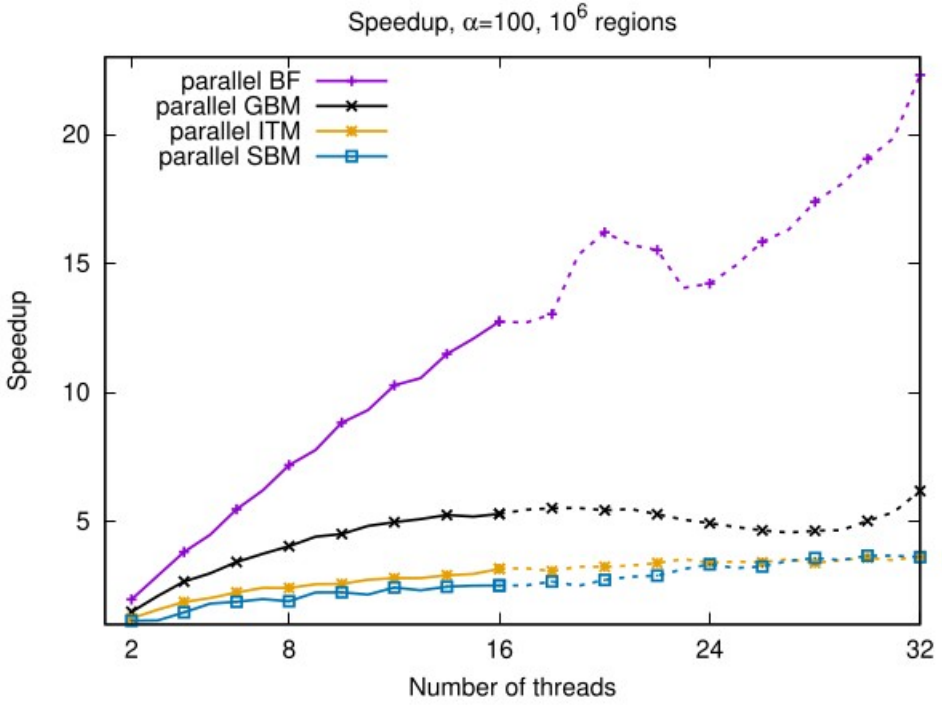
```
#pragma omp parallel for
for (int i=0; i<n; i++)
 for (int j=0; j<n; j++)
  for (int k=0; k<n; k++)
   C[i][j] += A[i][k] * B[k][j]
```

Example

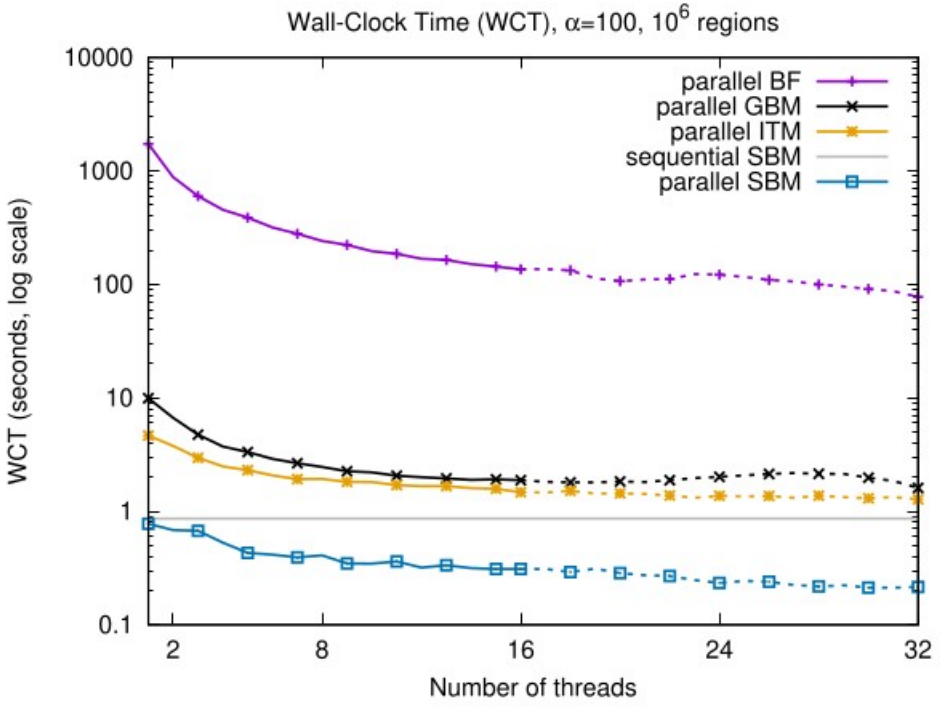
- See omp-matmul.c
  - demo-strong-scaling.sh computes the times required for strong scaling (demo-strong-scaling.ods).
  - demo-weak-scaling.sh computes the times required for weak scaling (demo-weak-scaling.ods).

Speedup alone is not always meaningful by itself

- A program can be more scalable and less efficient than another program!
- Example:



(b) Speedup



(a) Wall-clock time

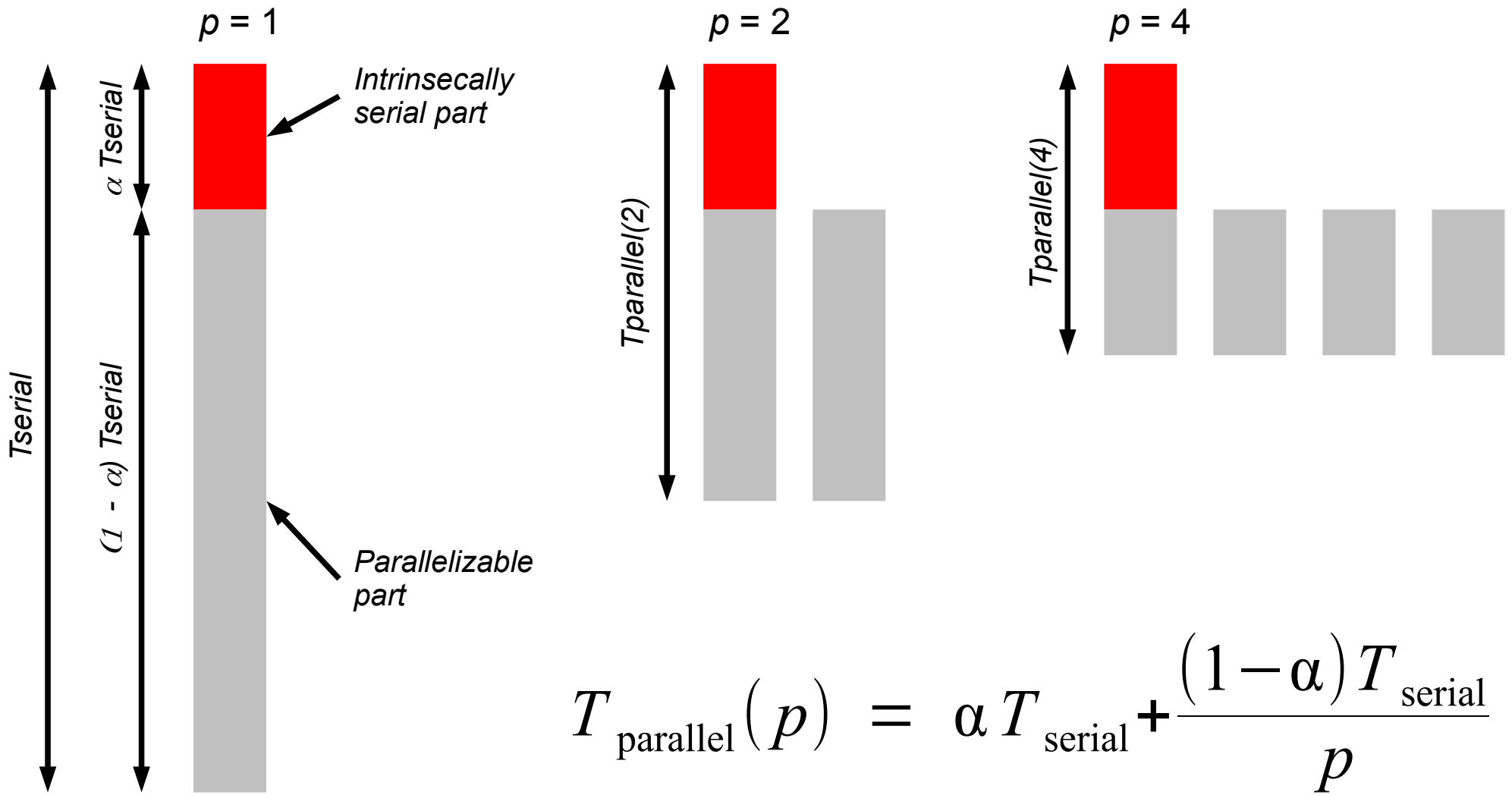
Theoretical bounds on scalability

Amdahl's scaling law

- Suppose that a fraction α of the total execution time of the serial program can not be parallelized.
  - E.g., due to:
    - Algorithmic limitations (data dependencies).
    - Bottlenecks (e.g., shared resources).
    - Startup overhead.
    - Communication costs.
    - Scheduling overhead.
- Suppose that the remaining fraction (1 - α) can be fully parallelized.
- Then, we have:

$$T_{\text{parallel}}(p) = \alpha T_{\text{serial}} + \frac{(1-\alpha) T_{\text{serial}}}{p}$$

Example



$$T_{\text{parallel}}(p) = \alpha T_{\text{serial}} + \frac{(1-\alpha) T_{\text{serial}}}{p}$$

Example

- Suppose that a program has T_{serial} = 20s.
- Assume that 10% of the time is spent in a serial portion of the program.
- Therefore, the execution time of a parallel version with *p* processors is:

$$T_{\text{parallel}}(p) = 0.1T_{\text{serial}} + \frac{0.9T_{\text{serial}}}{p} = 2 + \frac{18}{p}$$

Example (cont.)

- The speedup is

$$S(p) = \frac{T_{\text{serial}}}{0.1 \times T_{\text{serial}} + \frac{0.9 \times T_{\text{serial}}}{p}} = \frac{20}{2 + \frac{18}{p}}$$

- What is the maximum speedup that can be achieved when p $\rightarrow +\infty$?

Amdahl's Law

- What is the maximum speedup?

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)}$$

$$= \frac{T_{\text{serial}}}{\alpha T_{\text{serial}} + \frac{(1-\alpha) T_{\text{serial}}}{p}}$$

$$= \frac{1}{\alpha + \frac{1-\alpha}{p}}$$

Amdahl's Law

Gene Myron Amdahl
(1922—2015)

Gene M. Amdahl. *Validity of the single processor approach to achieving large scale computing capabilities*. Proc. Spring joint computer conference (AFIPS '67), April 18-20, 1967, pp. 483–485.
https://doi.org/10.1145/1465482.1465560

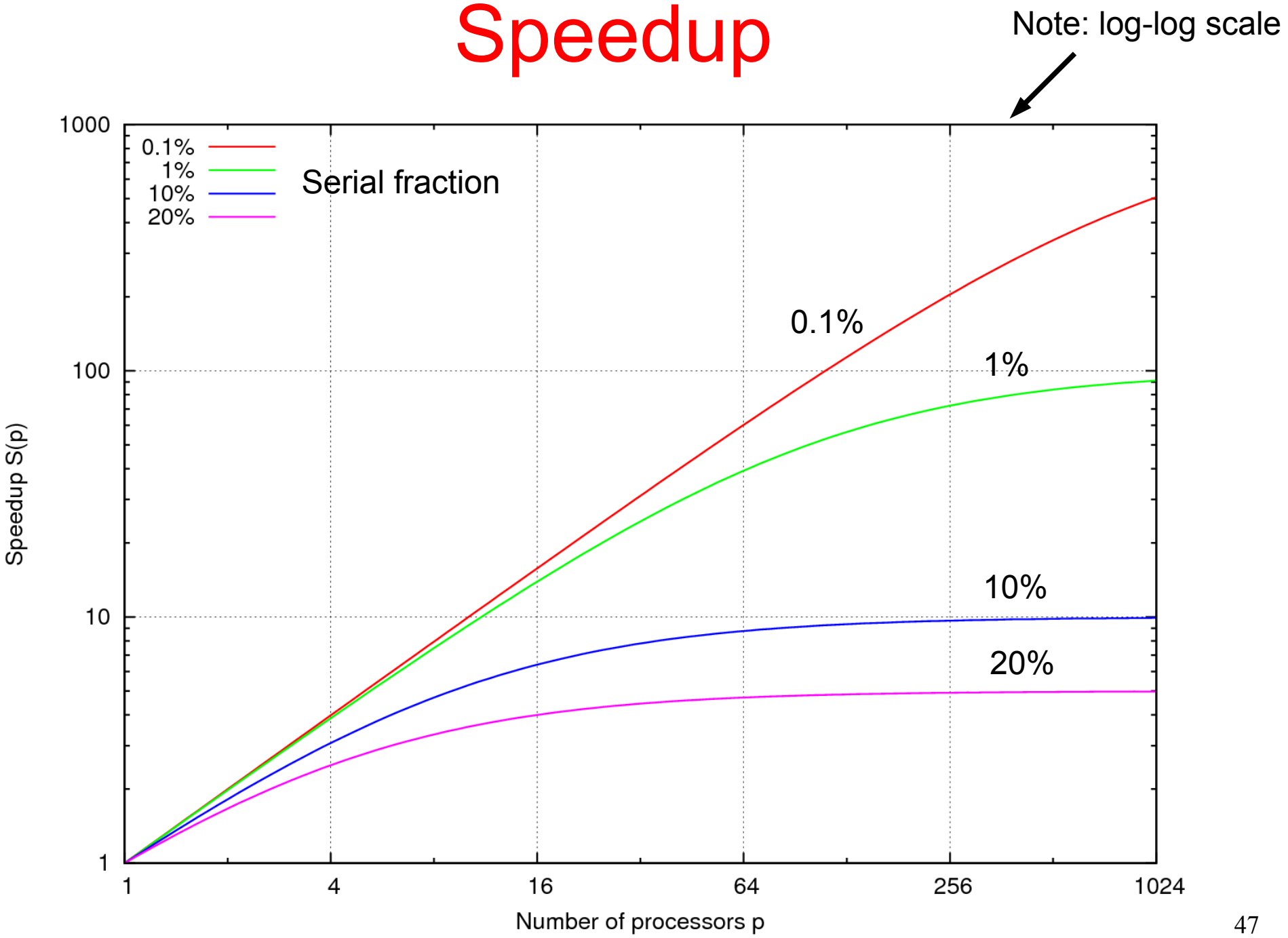
Amdahl's Law

- From $$S(p) = \frac{1}{\alpha + \frac{(1-\alpha)}{p}}$$

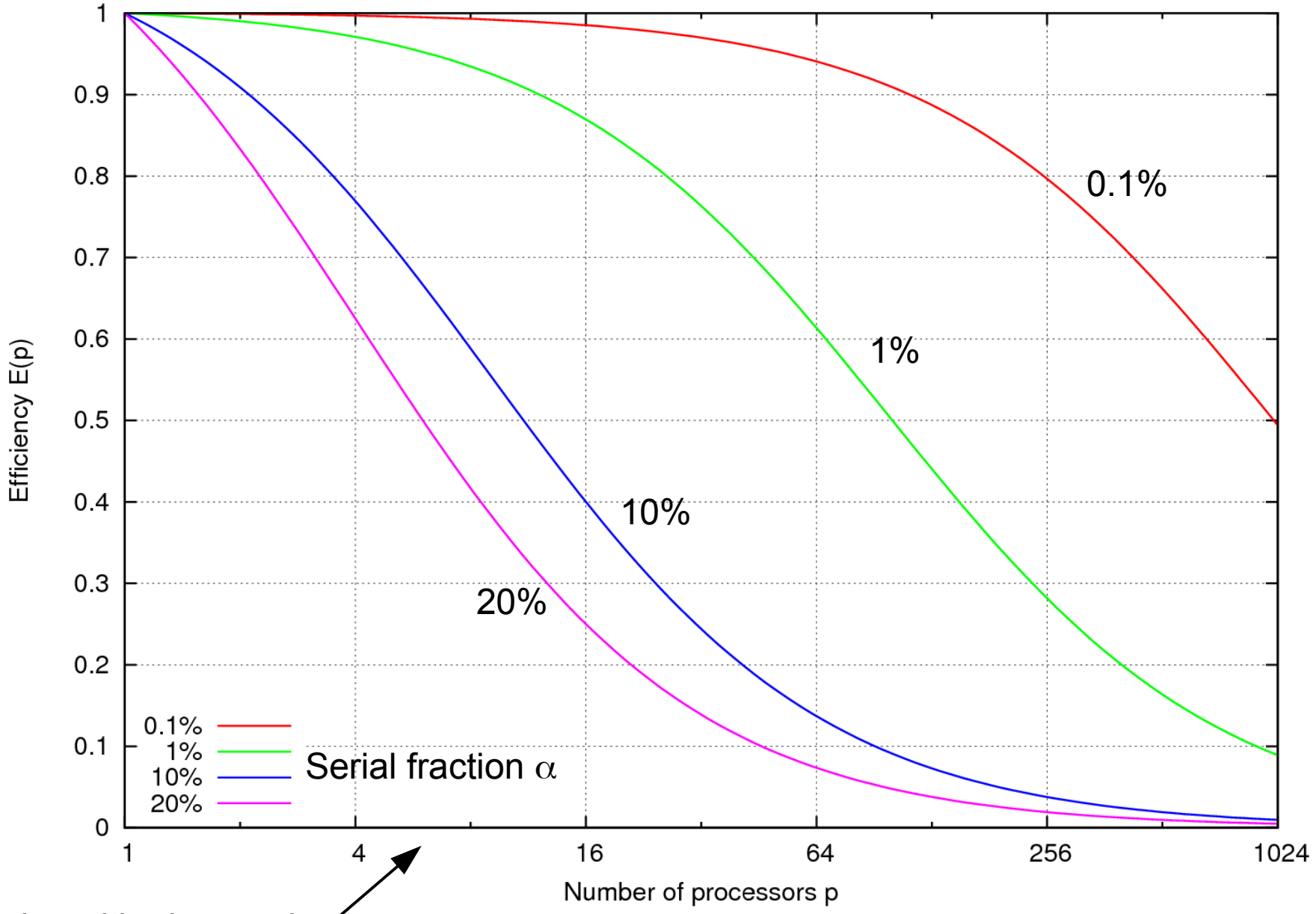
  we get an asymptotic speedup 1 / α when *p* grows to infinity.

> **Amdahl's Law**: If a fraction α of the total execution time is spent on a serial portion of the program, then the maximum achievable speedup is 1/α.

Speedup


Note: log-log scale
Serial fraction
0.1%
1%
10%
20%
1000
100
10
1
Speedup S(p)
1
4
16
64
256
1024
Number of processors p

Strong Scaling Efficiency and Amdahl's Law



Note: logarithmic *x* scale

Concluding remarks

- The performance of parallel programs is measured by its scalability, i.e., ability to either
  - Solve the same problem in less time using more processors (strong scaling efficiency), or
  - Solve a larger problem using more processors (weak scaling efficiency).
- The Work/Span cost model is widely used to analyze the theoretical scalability of parallel programs.
- The Speedup, Strong and Weak scaling efficiencies are used to measure the actual scalability of running programs.